Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 16

Heaps

And Heap Sort

Last time we saw...

- A nearly complete binary tree can be implemented using an array of values
  - ```
    int Left(int i, int N) { return (2*i+1)>N-1 ? -1:(2*i+1); }
    int Right(int i, int N) { return (2*i+2)>N-1 ? -1:(2*i+2); }
    int Parent(int i) { return (i==0) ? -1:(i-1)/2; }
    ```
- A heap is a nearly complete binary tree with the heap property
  - So a heap can be implemented using an array!
    ```

Heap in an Array

```
struct heap {
  std::vector<int> Keys; // the actual data
  int Left(int i) { return (2*i+1) < Keys.size() ? (2*i+1) : -1; }
  int Right(int i) { return (2*i+2) < Keys.size() ? (2*i+2) : -1; }
  int Parent(int i) { return (i>0) ? (i-1)/2 : -1; }
  void bubbleUp(int i, int newkey);
  void bubbleDown(int i, int newkey);
  void push(int x) {Keys.push_back(0); bubbleUp(Keys.size()-1,x);}
  void pop() { // assumes heap not empty
    int x = Keys.back(); Keys.pop_back(); if(!Keys.empty()) bubbleDown(0,x);
  }
  int top() { return Keys[0]; } // assumes heap not empty
 /* more functions */
};
```

Heap in an Array

```
void heap::bubbleUp(int actv, int newkey) {
  for(; Parent(actv)!=-1 && Keys[Parent(actv)] > newkey; actv = Parent(actv);)
    Keys[actv] = Keys[Parent(actv)];
  Keys[actv] = newkey; //outside the loop: newkey copied into actv only now
}
void heap::bubbleDown(int actv, int newkey) {
  for(bool done=false; !done; ) {
    int smallest = newkey;
    if(Left(actv)!=-1 && Keys[Left(actv)]<smallest) smallest = Keys[Left(actv)];
    if(Right(actv)!=-1 && Keys[Right(actv)]<smallest) smallest = Keys[Right(actv)];
    if(smallest==newkey) done = true;
    else { // bubble the smaller child up (left child guaranteed to exist)
      Keys[actv] = smallest;
      actv = (smallest==Keys[Left(actv)]) ? Left(actv) : Right(actv);
    }
  }
  Keys[actv] = newkey; //outside the loop: newkey copied into actv only now
}
```

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!

Heap vs. BST

- BST organises the data more: Essentially sorts the data (since sorted order can be retrieved in O(n) time by in-order traversal). O(log n) search in a balanced BST.
- Heap maintains enough information to only access/remove the minimum. Provides no search facility (other than O(n) scanning).
  - What do heaps offer in return?
    - Uses only O(1) extra space beyond the data: Data is held in its original container (provided it supports random access)
      - If we just need a priority queue, a BST is a waste of space
    - O(n) time construction from a vector (coming up)
      - If we are not planning to dequeue all the elements, constructing a BST (in time $\Theta(n \log n)$) is a waste of time

Make-Heap

- Can make a heap, in O(n) time

```
struct heap {
  std::vector<int> Keys; // the actual data
  /* ... */
  void bubbleDown(int i, int newkey);

  void makeHeap(int root = 0) { // a recursive version
    if(root==-1) return;
    makeHeap(Left(root)); makeHeap(Right(root));
    bubbleDown(root, Keys[root]);
  }

  heap(std::vector<int> v) : Keys(v) { if(Keys.size()>0) makeHeap(); }
};
```

Make-Heap

- Can make a heap, in O(n) time
- Why O(n)?
  - Every node is visited once (like in a post-order traversal): O(n) for this
  - From every node, bubbleDown is called
    - O $\left(\sum_{\text{node}} \text{height(node)} \right)$ for this
    - At *depth* d, 2^d nodes. i.e, at *height* i, 2^{h-i} nodes (h = height of root)
    - $\sum_{\text{node}} \text{height(node)} \leq \sum_{i=0}^{h} i2^{h-i} = 2^h \sum_{i=0}^{h} \frac{i}{2^i} = O(2^h).$
    - Again, O(n)

```
void makeHeap(int root = 0) {
  if(root==-1) return;
  makeHeap(Left(root));
  makeHeap(Right(root));
  bubbleDown(root, Keys[root]);
}
```

Make-Heap: Non-Recursive

- Can we eliminate recursion, and use only O(1) additional space?
- Just need that before bubbleDown is called on a node, its two children have been processed (and are roots of valid heaps)
- Any child-before-parent order suffices (post-order traversal being one such)
- A simple child-before-parent order is the reverse of the "breadth-first" order in which the data is maintained
  - For index ≤ n/2, no left child (at n/2, 2*(n/2)+1 = n or n+1) or right child, and no need to call `bubbleDown`

```
void makeHeap(int root = 0) {
  if(root==-1) return;
  makeHeap(Left(root));
  makeHeap(Right(root));
  bubbleDown(root, Keys[root]);
}
```

```
void makeHeap() { //assumes Keys not empty
  for(int i = Keys.size()/2-1; i >= 0; --i)
    bubbleDown(i, Keys[i]);
}
```

Heap Sort

- An *in-place* sorting algorithm running in O(n log n) time
- Two modifications to our heap:
  - max-heap (bigger elements at the top of the heap)
  - We need to be able to set the heap to be any prefix of the array (specifically, `Left()` and `Right()` used in `bubbleDown()` take heap-size as input)
- In-place sorting:
  - Given an array `A` of values, in O(n) time and using O(1) extra space, make a heap out of them (using the non-recursive version of `makeHeap()`)
  - Repeat n-1 times:
    - `A[0]` has the maximum element. Swap `A[0]` and `A[n-1]` and set the size of the heap to be n-1 (so that `A[n-1]` is outside the heap).
    - Call `bubbleDown()` on the root with the value in `A[0]` as the new key